

TENSOR NETWORKS PARA SIMULAÇÃO DE CIRCUITOS QUÂNTICOS

Alexandre Silva

Roteiro:

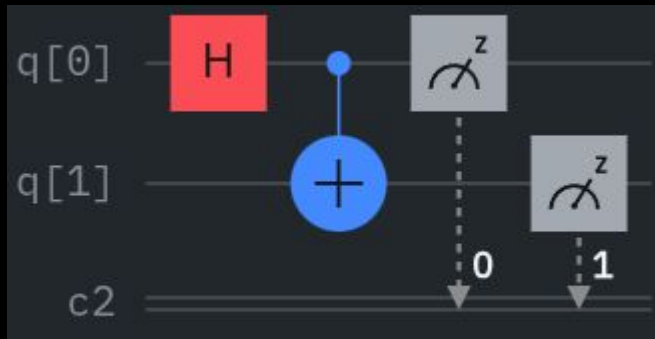
1. Contextualização sobre computação quântica;
2. Entendendo o problema;
3. Entendendo Tensor Networks;
4. Algoritmo em si + Análises.

O que é a Computação Quântica?

- Utilizar a Mecânica Quântica para realizar computações;
- Átomos, partículas, fótons, elétrons, etc;
- Possibilidade de avanços;
 - Medicina, Farmácia, Criptografia, Machine Learning, etc.

Modelo de Computação Quântica

- Modelo de Circuito é o mais usado;
- Portas são matrizes (operadores);
- Linhas são qubits (vetores);
- Matrizes evoluem o estado (vetor).



Fonte: Autoria própria via [IBM Quantum Composer](#)

$$|\psi\rangle = \begin{bmatrix} \alpha_{00} \\ \alpha_{01} \\ \alpha_{10} \\ \alpha_{11} \end{bmatrix}$$

Fonte: [ResearchGate](#)

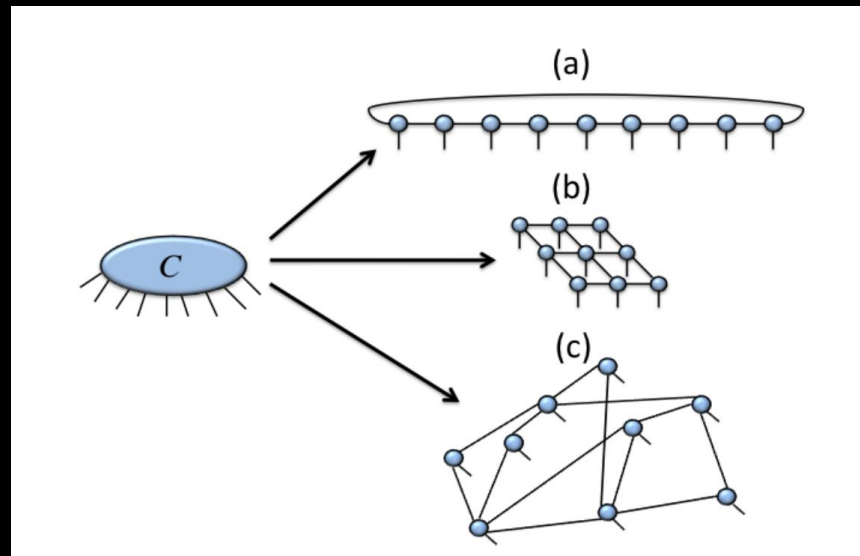
Problema

- Computadores Quânticos ainda são limitados - Noisy Intermediate-Scale Quantum (NISQ);
- Solução → Simulações para validar os algoritmos (HPC);
- Simulações são custosas;
 - Possibilidade de estados cresce a $N = 2^n$;
 - 40 qubits → $2^{40} \times 2^3$ bytes (float) = 2^{43} bytes = 8Tb.

$$|\Psi\rangle = \sum_{i_1 i_2 \dots i_N} C_{i_1 i_2 \dots i_N} |i_1\rangle \otimes |i_2\rangle \otimes \dots \otimes |i_N\rangle$$

Tensor Networks

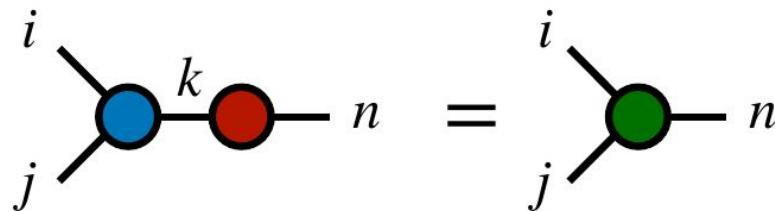
- Tensor $\rightarrow$ Generalização de escalares, vetores, matrizes, etc;
- Aplicar operações conforme necessário;
- Tensors menores consomem menos memória;
- Paralelizáveis.



Fonte: [arXiv:1306.2164](https://arxiv.org/abs/1306.2164)

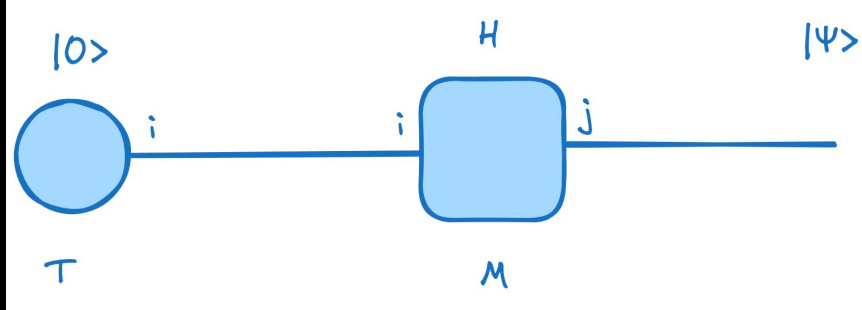
Principal Operação

- Contrações (Unir tensors);
 - Sequência de contrações modifica o custo;
 - Mais índices mais operações e dimensões.



$$\sum_k T_{ijk} M_{kn} = R_{ijn}$$

Fonte: [arXiv:2007.14822](https://arxiv.org/abs/2007.14822)



$$T = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad M = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$Q_j = \sum_i T_i M_{ij}$$

$$Q_0 = T_0 M_{00} + T_1 M_{10} = \frac{1}{\sqrt{2}} ((1)(1) + (0)(1))$$

$$Q_1 = T_0 M_{01} + T_1 M_{11} = \frac{1}{\sqrt{2}} ((1)(1) + (0)(-1))$$

$$Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Fonte: Criação própria via [Excalidraw](#) e [Overleaf](#)

Custo de operações

- Medido em FLOPs;
- Physical Dimensions (d);
 - Valores binários do sistema;
 - Constante;
- Bond dimensions (D);
 - Quantidade de entanglements (ligações);
 - Quanto mais melhor a aproximação do estado, mas mais custoso $\rightarrow$ cresce 2^D ;
 - Contrações alteram o tamanho.

$$Q_0 = T_0 M_{00} + T_1 M_{10} = \frac{1}{\sqrt{2}}((1)(1) + (0)(1))$$
$$Q_1 = T_0 M_{01} + T_1 M_{11} = \frac{1}{\sqrt{2}}((1)(1) + (0)(-1))$$

$$d = 2$$

$$O(d) * O(d) = O(d^2)$$

$$\text{FLOPs} = d(2d - 1)$$

Passos para Simular o Circuito

1. Definição da rede (circuito);
2. Simplificação da rede;
3. Busca pelo melhor caminho de contração;
4. Aplicação das contrações → resultado.

Obs: Grafos são dispersos (circuitos não são completamente interligados) e networks de formatos desconhecidos.

1 Simplificação

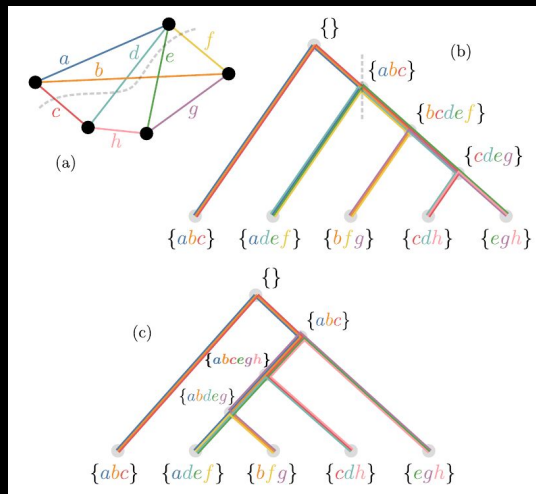
- Depende da estrutura da rede;
- Simplificações → Absorção de tensors, SVD, etc. Tidos como $\approx O(1)$ (tempo);
- Complexidade tempo:
 - $O(d \times n) \approx O(n)$;
 - d (passos) é pequeno;
 - nodes n é mais expressivo;
- Espaço:
 - Armazenamento reduz a cada iteração.

```
while there are simplifications do
  for node in graph do
    Simplify1(node)
    Simplify2(node)
    Simplify3(node)
    ...
  end for
end while
```

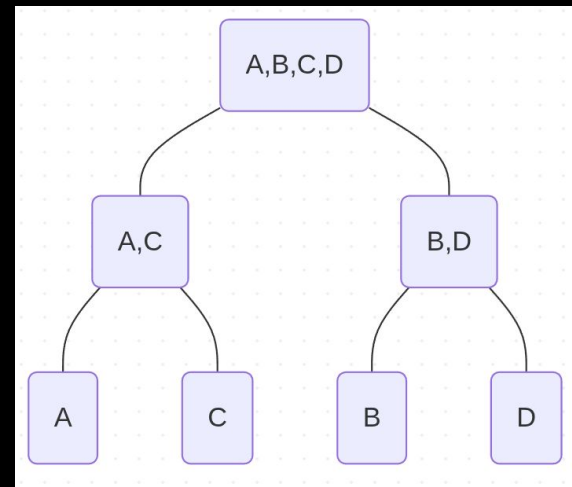
Fonte: Criação própria via [Overleaf](#)

2 Busca pelo melhor caminho de contração (Hyper-Par)

- Repartição de subárvores;
- Algoritmos ótimos nas subárvores (Optimal/Greedy) → Problemas menores;
- Parâmetros de balanceamento (ϵ), repartições (k) e cut-off;
 - Sample de parâmetros.



Fonte: [arXiv:2002.01935v4](https://arxiv.org/abs/2002.01935v4)



Fonte: Criação própria via [Mermaid](#)

2.1 Hyper-Par Algoritmo

- Complexidade Tempo:
 - $O(k * \epsilon * \text{cut} * (V + \text{cost}))$;
 - $O(V + \text{cost})$;
 - Partition é a parte mais custosa;
 - Custo é $O(n)$ → calcula o custo de cada nó e soma;
- Complexidade Espaço:
 - $O(n)$ → tamanho do grafo.

```
for each combination  $(k, \epsilon, \text{cut})$  do
   $V \leftarrow \text{Partition}(G, \text{cut}, \epsilon, k)$ 
   $\text{cost} \leftarrow \text{Cost}(V)$ 
  if  $\text{cost} < B$  then
     $B \leftarrow \text{cost}$ 
     $BG \leftarrow V$ 
  end if
end for
```

Fonte: Criação própria via [Overleaf](#)

2.2 Partição

- Criação da árvore via recursão (bipartição);
- Complexidade Tempo:
 - KaHyPar sempre executa;
 - Opt é custoso (quando n grande);
 - $j^* O(\text{KaHyPar}) + h^* O(\text{Opt})$
 - $O(\text{KaHyPar})$;
- Complexidade Espaço:
 - $O(n) \rightarrow \text{children}$ (todos os nós);

```
function PARTITION( $V, \text{cut}, \epsilon, k$ )  
  if  $|V| \leq 1$  then  
    return Leaf( $V$ )  
  end if  
  if  $|V| \leq \text{cut}$  then  
    return Opt( $V$ )  
  end if  
  
  parts  $\leftarrow$  KaHyPar( $V, \epsilon, k$ )  
  children  $\leftarrow \{\}$   
  
  for each  $V_i$  in parts do  
    children.append(Partition( $V_i, \text{cut}, \epsilon, k$ ))  
  end for  
  return Node( $V, \text{children}$ )  
end function
```

Fonte: Criação própria via [Overleaf](#)

2.3 Hyper-Greedy Opt.

- Calcula o custo de cada nó;
- Probabilidade de escolher um pior;
- Complexidade tempo:
 - $O(n^3)$;
- Complexidade espaço:
 - $O(n^2) \rightarrow$ pares;
- n deve ser pequeno.

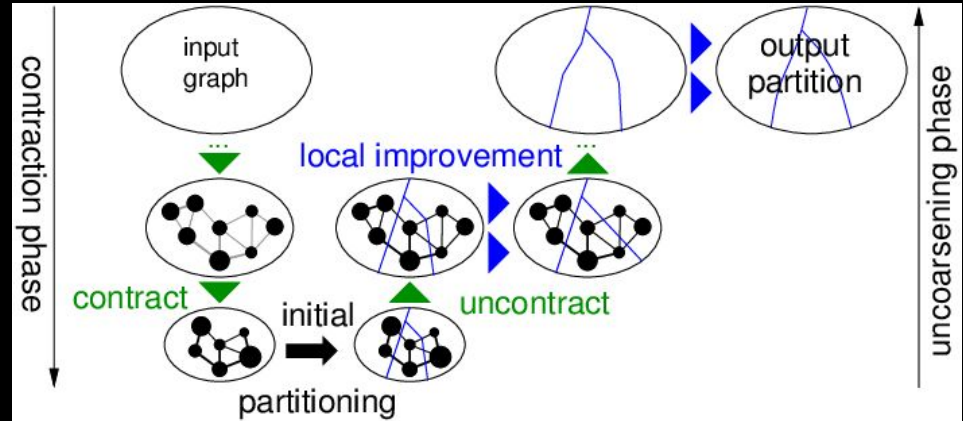
```
function OPT(V)
  outNodes  $\leftarrow \{\}$ 
  nodes  $\leftarrow$  every item in  $V$  as Leaf
  while len(nodes) > 1 do
    pairs  $\leftarrow \{\}$ 
    costs  $\leftarrow \{\}$ 

    for each pair (i, j) in nodes do
      cost  $\leftarrow$  size(nodei) -  $\alpha$ (size(nodei) + size(nodej))
      pairs.append((i, j))
      costs.append(cost)
    end for

    probs  $\leftarrow \{\}$ 
    Z  $\leftarrow 0$ 
    for cost in costs do
      p  $\leftarrow \exp(-cost/\tau)$ 
      probs.append(p)
      Z  $\leftarrow Z + p$ 
    end for

    for k in range(len(probs)) do
      probsk  $\leftarrow$  probsk/Z
    end for
    i, j  $\leftarrow$  randomChoice(pairs, probs)
    newNode  $\leftarrow$  Node(nodesi, nodesj)
    remove nodesi, nodesj from nodes
    outNodes.append(newNode)
  end while
  return outNodes
end function
```

- Multilevel graph partitioning (KaHyPar);
 1. Reduz o grafo;
 2. Particiona;
 3. Realiza o caminho inverso;
- Na prática:
 - Usa múltiplos algoritmos (Portifólio);
 - Hypergraphs;
 - Pré-processamento e otimização.



Fonte: [ResearchGate](#)

2.4.1 Auxiliares

$$\text{Tempo} = O(e_n v_e)$$

$$\text{Espaço} = O(g)$$

$$\text{Tempo} = O(e_s = e_n + e_g)$$

$$\text{Espaço} = O(e_s)$$

$$\text{Tempo} = O(e_g)$$

Espaço $\rightarrow$ Diminui

```
function NEIGHBORSLOCALCLUSTERS(node)
  neighbors  $\leftarrow \{\}$ 
  for edge in incidentEdgesnode do
    if len(edge.vertices)  $\leq$  threshold then

      for neighbor in edge.vertices do
        if neighbor  $\neq$  node then
          neighbors.add(neighbor)
        end if
      end for

    end if
  end for

  return neighbors
end function
```

```
function RATE(node, neighbor)
  s  $\leftarrow$  incidentEdgesnode  $\cap$  incidentEdgesneighbor
  return  $\sum_e^s 1/\text{len}(e.\text{vertices}) - 1$ 
end function
```

```
function CONTRACT(node, neighbor)
  node.weight  $\leftarrow$  node.weight + neighbor.weight
  for edge in incidentEdgesneighbor do
    edge.removeVertex(neighbor)
    if ! node in edge.vertices then
      edge.addVertex(node)
      node.addIncidentEdge(edge)
    end if

    if len(edge.vertices) < 2 then
      removeEdge(edge)
    end if
  end for
  removeVertex(neighbor)
end function
```

2.4.2 Coarsening

- Shuffle gera resultados diferentes;
- Procura por pequenos clusters.

n =nodes; g =neighbors; e =edges; v =vertices;

- $O(n * (e_n v_e + g e_s + e_g))$

Todos os nós conectados (threshold alto):

- $e_n = n$; $v_e = n$; $g = n-1$; $e_s = n$; $e_g = n$;
- $O(n^3)$;

Caso mais real:

- Todos os valores são pequenos;
- Tempo $\rightarrow O(n)$;
- Espaço $\rightarrow O(n)$.

```
nodes ← shuffle(G.nodes)
matched ← {}

for node in nodes do
  if node in matched then
    continue
  end if
  bestNeighbor ← null
  maxRating ← -∞

  neighbors ← neighborsLocalClusters(node)
  for neighbor in neighbors do
    if neighbor == node || neighbor in matched || weightnode +
weightneighbor > maxVertexWeight then
      continue
    end if

    r ← rate(node, neighbor)
    if r > maxRating then
      maxRating ← r
      bestNeighbor ← neighbor
    end if
  end for

  if bestNeighbor then
    contract(node, bestNeighbor)
    matched.add(node, bestNeighbor)
  end if
end for
```

2.4.4 Bipartitioning

Tempo = $O(n \log n)$

Espaço = $O(n)$

Tempo $\approx O(n \log n)$
(Considerando
coarsening $\approx 1/2^k$), na
prática $\approx O(n)$
Espaço = $O(n)$

Tempo = $O(n)$
Espaço = $O(n)$

```
function RECURSIVEBIPARTITION( $G, k$ )
  if  $k \leq 1$  then:
    return {}
  end if
   $V_0, V_1 \leftarrow \text{2wayPartition}(G)$ 
   $G_0 \leftarrow$  new graph with only  $V_0$ 
   $G_1 \leftarrow$  new graph with only  $V_1$ 
  left  $\leftarrow$  recursiveBipartition( $G_0, k/2$ )
  right  $\leftarrow$  recursiveBipartition( $G_1, k/2$ )
  return left + right
end function

function 2WAYPARTITION( $G$ )
  hierarchy  $\leftarrow \{G\}$ 
  currentG  $\leftarrow G$ 
  while currentG.numVertices > threshold do
    newG  $\leftarrow$  coarse(currentG)
    hierarchy.append(newG)
    currentG  $\leftarrow$  newG
  end while
  cut  $\leftarrow$  partition(currentG)

  while hierarchy do
    coarsedG  $\leftarrow$  hierarchy.pop()
    cut  $\leftarrow$  projectCut(cut, coarsedG)
  end while
  return cut
end function

function PROJECTCUT( $cut, G$ )
  newPartition  $\leftarrow \{\}$ 
  for node in  $G$ .nodes do
    SU  $\leftarrow$  node.parent
    newPartitionnode  $\leftarrow$  cutSU
  end for
  return newPartition
end function
```

```
function PARTITION( $G$ )
  bestSize  $\leftarrow \infty$ 
  bestPartition  $\leftarrow$  null

  for attempt in range(attempts) do
     $rand_0, rand_1 \leftarrow$  Two random nodes from  $G$ 
     $block_0 \leftarrow \{rand_0\}$ 
     $block_1 \leftarrow \{rand_1\}$ 

     $candidates_0 \leftarrow$  PriorityQueue( $rand_0$ .neighbors)
     $candidates_1 \leftarrow$  PriorityQueue( $rand_1$ .neighbors)
    targetWeight  $\leftarrow G$ .totalWeight/2

    while  $len(block_0) + len(block_1) < G$ .vertices do
      currentBlock  $\leftarrow$  Current Smaller block
      currentQueue  $\leftarrow$  Queue relative to the current block

      if ! currentQueue.empty then
        maxNode  $\leftarrow$  currentQueue.popMaxGainNode()
        if ! maxNode in  $block_0$  && ! maxNode in  $block_1$  then
          currentBlock.add(maxNode)

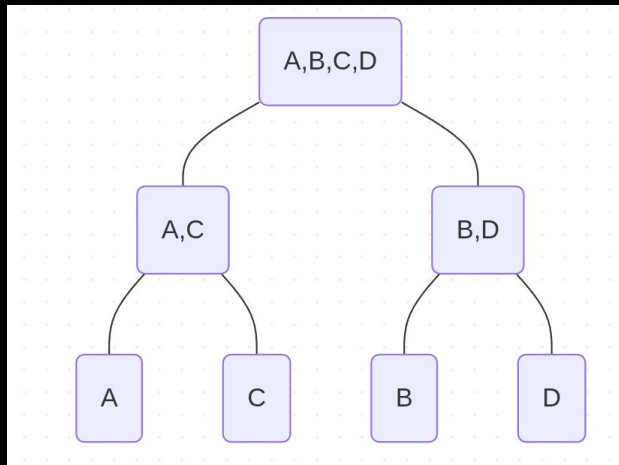
          for neighbor in maxNode.neighbors do
            if ! neighbor in  $block_0$  && ! neighbor in  $block_1$  then
              currentQueue.insert(neighbor)
            end if
          end for
        end if
      else
        currentBlock.add( $G$ .getRandomUnassignedNode())
      end if
    end while

    partition  $\leftarrow (block_0, block_1)$ 
    if partition.cutSize < bestSize then
      bestSize  $\leftarrow$  partition.cutSize
      bestPartition  $\leftarrow$  partition
    end if
  end for
  return bestPartition
end function
```

Tempo = $O(n)$
Espaço = $O(n)$

3 Contração

- A contração segue a árvore criada;
- Tempo $\rightarrow O(n * 2^D)$;
- Espaço $\rightarrow O(2^D)$ armazenamento dos tensors (possuem tamanhos diferentes).



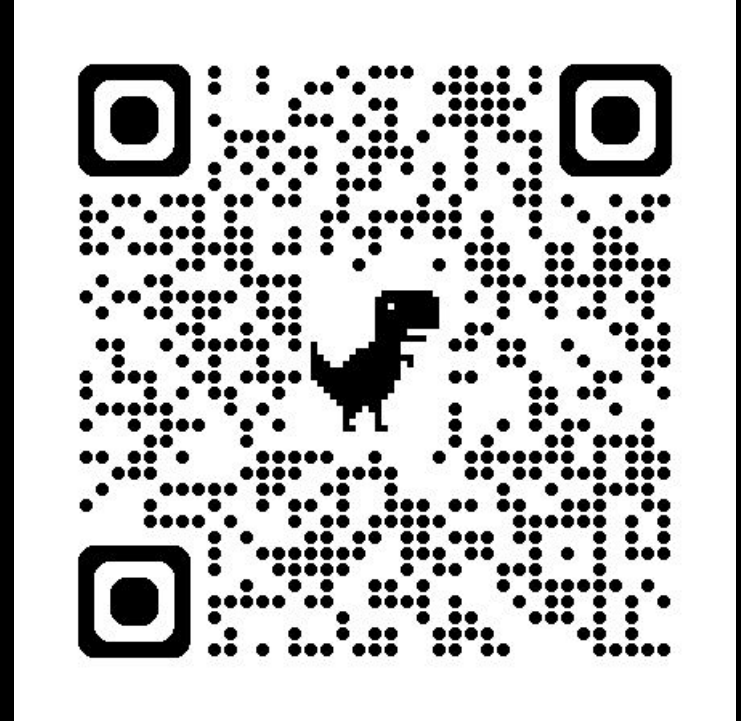
```
function CONTRACTNET(node)
  if node is a Leaf then
    return contractLocal(node.tensors)
  end if
  leftTensor  $\leftarrow$  contractNet(node.left)
  rightTensor  $\leftarrow$  contractNet(node.right)
  return contract(leftTensor, rightTensor)
end function
```

Considerações Finais

- Caminho $\rightarrow$ limitante é o KaHyPar $\approx O(n \log n)$;
- Contração $\rightarrow$ limitante é a dimensão $D \approx O(n * 2^D)$;
- Espaço $\rightarrow O(2^D)$, quando D é pequeno $\rightarrow O(n)$;
- Em casos reais (melhorias):
 - Uso de Paralelização e aceleradores;
 - Uso de múltiplas funções de otimização.

Repositório

[https://github.com/Dpbm/
mestrado-aulas/tree/main/
algoritmos/projeto-seminar
io](https://github.com/Dpbm/mestrado-aulas/tree/main/algoritmos/projeto-seminario)



Referências

- FISHMAN, M.; WHITE, S.; STOUDENMIRE, E. The ITensor Software Library for Tensor Network Calculations. SciPost Physics Codebases, p. 004, 23 ago. 2022.
- Overview — NVIDIA cuQuantum 23.06.0 documentation. Disponível em: <<https://docs.nvidia.com/cuda/cuquantum/23.06.0/cutensornet/overview.html>>. Acesso em: 4 abr. 2026.
- SCHINDLER, F.; JERMYN, A. S. Algorithms for tensor network contraction ordering. Machine Learning: Science and Technology, v. 1, n. 3, p. 035001, 2 jul. 2020.
- BRIDGEMAN, J. C.; CHUBB, C. T. Hand-waving and interpretive dance: an introductory course on tensor networks. v. 50, n. 22, p. 223001–223001, 9 mar. 2016.
- NGUYEN, T. et al. Tensor Network Quantum Virtual Machine for Simulating Quantum Circuits at Exascale. ACM Transactions on Quantum Computing, v. 4, n. 1, p. 1–21, 21 out. 2022.
- ORÚS, R. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. Annals of Physics, v. 349, p. 117–158, out. 2014.
- GRAY, J.; KOURTIS, S. Hyper-optimized tensor network contraction. Quantum, v. 5, p. 410, 15 mar. 2021.
- SCHLAG, S. et al. High-Quality Hypergraph Partitioning. ACM Journal of Experimental Algorithmics, v. 27, p. 1–39, 21 abr. 2022.
- INACIO, Y. R. Tensor Networks: Foundations, Architectures, and Practical Implementation. Disponível em: <<https://medium.com/@yara.rodrigues.inacio/tensor-networks-foundations-architectures-and-practical-implementation-ca7d146dd2c1>>. Acesso em: 5 maio. 2026.
- BRIDGEMAN, J. et al. Matrix Product States. Disponível em: <<https://quantumghent.github.io/TensorTutorials/3-MatrixProductStates/MatrixProductStates.html>>. Acesso em: 5 maio. 2026.